## IPD440 - APRENDIZAJE DE MAQUINAS

# Clase 4: Linear Models

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

# Modelos Lineales

Los siguientes son un conjunto de métodos destinados a la regresión en los que se espera que el valor objetivo sea una combinación lineal de las características. En notación matemática, si $\hat{y}$ es el valor predicho.

$$\hat{y}(w, x) = w_0 + w_1 x_1 + \ldots + w_p x_p$$

donde $w = (w_1, \ldots, w_p)$ son los coeficientes y $w_0$ el intercepto.

In [1]:
```
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
```

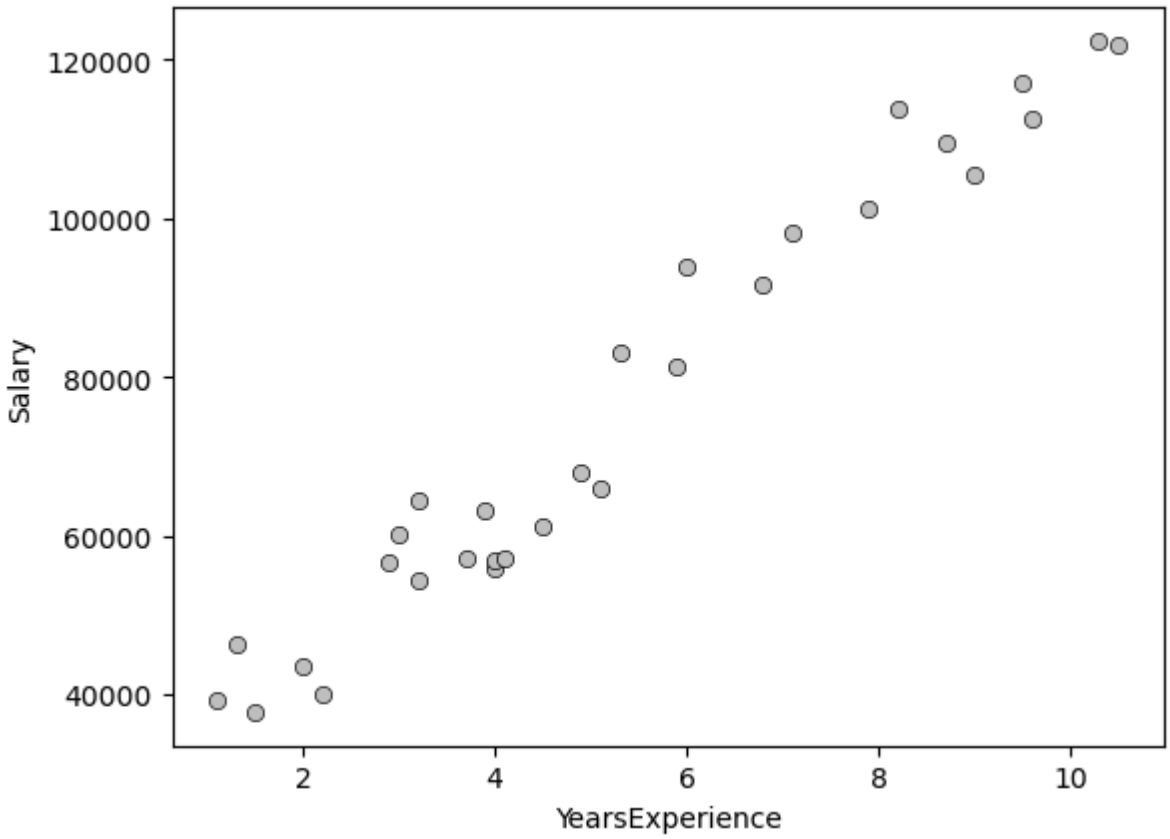
In [2]:
```
df_salary = pd.read_csv('Salary_Data.csv')
```

In [3]:
```
df_salary.head()
```

Out[3]:

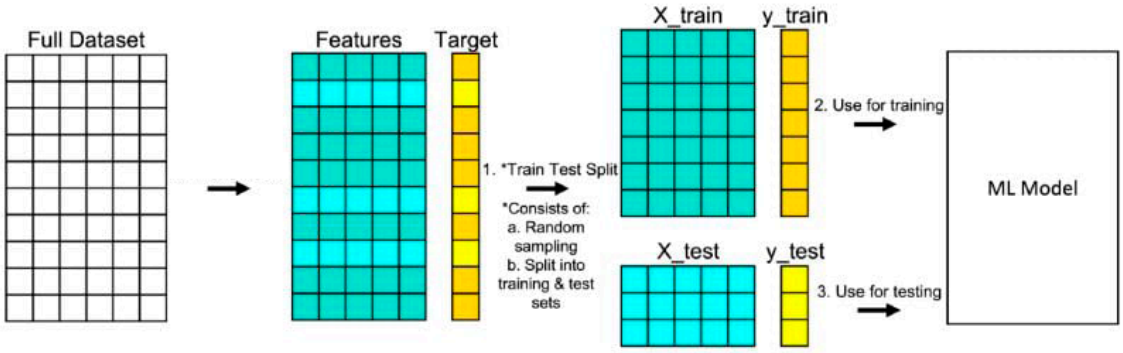
| | YearsExperience | Salary |
|---|---|---|
| **0** | 1.1 | 39343.0 |
| **1** | 1.3 | 46205.0 |
| **2** | 1.5 | 37731.0 |
| **3** | 2.0 | 43525.0 |
| **4** | 2.2 | 39891.0 |

In [4]:
```
sns.scatterplot(data=df_salary, x="YearsExperience", y="Salary", edgecolor='k', col
plt.show()
```




In [5]:
```
X_salary = df_salary[["YearsExperience"]]
y_salary = df_salary["Salary"]
```

# Train-Test-Split




In [6]:
```
from sklearn.model_selection import train_test_split
```

In [7]:
```
X_train_salary, X_test_salary, y_train_salary, y_test_salary = train_test_split(X_s
                                                                                 tes
```

In [8]:
```
plt.title('Salary vs Experience')
plt.ylabel('Salary')
plt.xlabel('Years of Experience')
plt.scatter(X_train_salary,y_train_salary, edgecolor='k', c='lightseagreen',label='
plt.scatter(X_test_salary,y_test_salary, edgecolor='k', c='aqua',label='Testing')
plt.legend()
plt.show()
```




## Mínimos cuadrados ordinarios

LinearRegression ajusta un modelo lineal con coeficientes $w = (w_1, \ldots, w_p)$ para minimizar la suma residual de cuadrados entre los objetivos observados en el conjunto de datos y los objetivos predichos por la aproximación lineal. Matemáticamente resuelve un problema de la forma:

$$\min_w ||Xw - y||_2^2$$

In [9]:
```
from sklearn import linear_model
```

In [10]:
```
regressor_salary = linear_model.LinearRegression()
regressor_salary.fit(X_train_salary, y_train_salary)
```

Out[10]:
```
▾ LinearRegression
LinearRegression()
```

In [11]:
```
print('Intercepto:',regressor_salary.intercept_)
print('Coeficientes:',regressor_salary.coef_)
print(f'y = {round(regressor_salary.intercept_)} + {round(regressor_salary.coef_[0]
```

```
Intercepto: 25321.583011776813
Coeficientes: [9423.81532303]
y = 25322 + 9424x
```

$$y = 25322 + 9424x$$

In [12]:
```
plt.title('Salary vs Experience')
plt.ylabel('Salary')
plt.xlabel('Years of Experience')
plt.scatter(X_train_salary, y_train_salary, edgecolor='k', c='lightseagreen',label=
plt.plot(X_train_salary, regressor_salary.intercept_ + regressor_salary.coef_ * X_t
         'b', linewidth=2, label='Linear Regression')
plt.legend()
plt.grid()
plt.show()
```




In [13]:
```
y_pred_salary = regressor_salary.predict(X_test_salary)
y_pred_salary
```

Out[13]:
```
array([115790.21011287,  71498.27809463, 102596.86866063,  75267.80422384,
        55477.79204548,  60189.69970699])
```

In [14]:
```
plt.title('Salary vs Experience')
plt.ylabel('Salary')
plt.xlabel('Years of Experience')
plt.scatter(X_train_salary, y_train_salary, edgecolor='k', c='lightseagreen',label=
plt.plot(X_train_salary, regressor_salary.intercept_ + regressor_salary.coef_ * X_t
         'b', linewidth=2, label='Linear Regression')
plt.grid()
plt.scatter(X_test_salary, y_pred_salary, marker='s', linewidth=2, facecolors='none
            zorder=100,s=80,label='y_pred')
plt.scatter(X_test_salary, y_test_salary, edgecolor='k', c='aqua',zorder=20,label='
plt.legend()
plt.show()
```




# Mean absolute error

Métrica de evaluación para problemas de regresión.

$$\mathbf{MAE} = \frac{\sum_{i=1}^{n} |\hat{y}_i - y_i|}{n},$$

donde $\hat{y}$ es `y_pred` e $y$ es `y_test` .

In [15]:
```
plt.title('Salary vs Experience')
plt.ylabel('Salary')
plt.xlabel('Years of Experience')
plt.scatter(X_train_salary, y_train_salary, edgecolor='k', c='lightseagreen',label=
plt.plot(X_train_salary, regressor_salary.intercept_ + regressor_salary.coef_ * X_t
         'b', linewidth=2, label='Linear Regression')
plt.grid()
plt.scatter(X_test_salary, y_pred_salary, marker='s', linewidth=2, facecolors='none
            zorder=100,s=80,label='y_pred')
plt.scatter(X_test_salary, y_test_salary, edgecolor='k', c='aqua',zorder=20,label='
plt.vlines(X_test_salary, y_test_salary, y_pred_salary, colors='r', linewidth=2, li
plt.legend()
plt.show()
```




In [16]:
```
from sklearn.metrics import mean_absolute_error
```

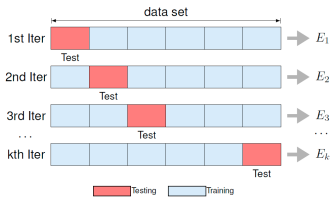
In [17]:
```
mae_salary = mean_absolute_error(y_test_salary,y_pred_salary)
print("Mean Absolute Error:",mae_salary)
```

```
Mean Absolute Error: 6286.453830757749
```

In [18]:
```
print(f"En otras palabras, el modelo se equivoca en promedio +/- USD ${round(mae_sa
```

```
En otras palabras, el modelo se equivoca en promedio +/- USD $6286.
```

# Cross-validation




In [19]:
```
from sklearn.model_selection import cross_val_score
```

In [20]:
```
scores = cross_val_score(regressor_salary, X_salary, y_salary, cv=5, scoring='neg_m
scores
```

Out[20]:
```
array([-4149.04416975, -4666.50514924, -7603.93320975, -6842.89022864,
       -5353.26254729])
```

In [21]:
```
np.mean(scores*-1)
```

Out[21]:
```
5723.127060934892
```

# Datos multidimensionales

In [22]:
```
df_cellphones = pd.read_csv("Cellphone.csv")
```

In [23]:
```
df_cellphones
```

Out[23]:

| | Product_id | Price | Sale | weight | resoloution | ppi | cpu core | cpu freq | internal mem | ram | RearCam | Fro |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 203 | 2357 | 10 | 135.0 | 5.20 | 424 | 8 | 1.350 | 16.0 | 3.000 | 13.00 | |
| **1** | 880 | 1749 | 10 | 125.0 | 4.00 | 233 | 2 | 1.300 | 4.0 | 1.000 | 3.15 | |
| **2** | 40 | 1916 | 10 | 110.0 | 4.70 | 312 | 4 | 1.200 | 8.0 | 1.500 | 13.00 | |
| **3** | 99 | 1315 | 11 | 118.5 | 4.00 | 233 | 2 | 1.300 | 4.0 | 0.512 | 3.15 | |
| **4** | 880 | 1749 | 11 | 125.0 | 4.00 | 233 | 2 | 1.300 | 4.0 | 1.000 | 3.15 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **156** | 1206 | 3551 | 4638 | 178.0 | 5.46 | 538 | 4 | 1.875 | 128.0 | 6.000 | 12.00 | |
| **157** | 1296 | 3211 | 8016 | 170.0 | 5.50 | 534 | 4 | 1.975 | 128.0 | 6.000 | 20.00 | |
| **158** | 856 | 3260 | 8809 | 150.0 | 5.50 | 401 | 8 | 2.200 | 64.0 | 4.000 | 20.00 | |
| **159** | 1296 | 3211 | 8946 | 170.0 | 5.50 | 534 | 4 | 1.975 | 128.0 | 6.000 | 20.00 | |
| **160** | 1131 | 2536 | 9807 | 202.0 | 6.00 | 367 | 8 | 1.500 | 16.0 | 3.000 | 21.50 | |

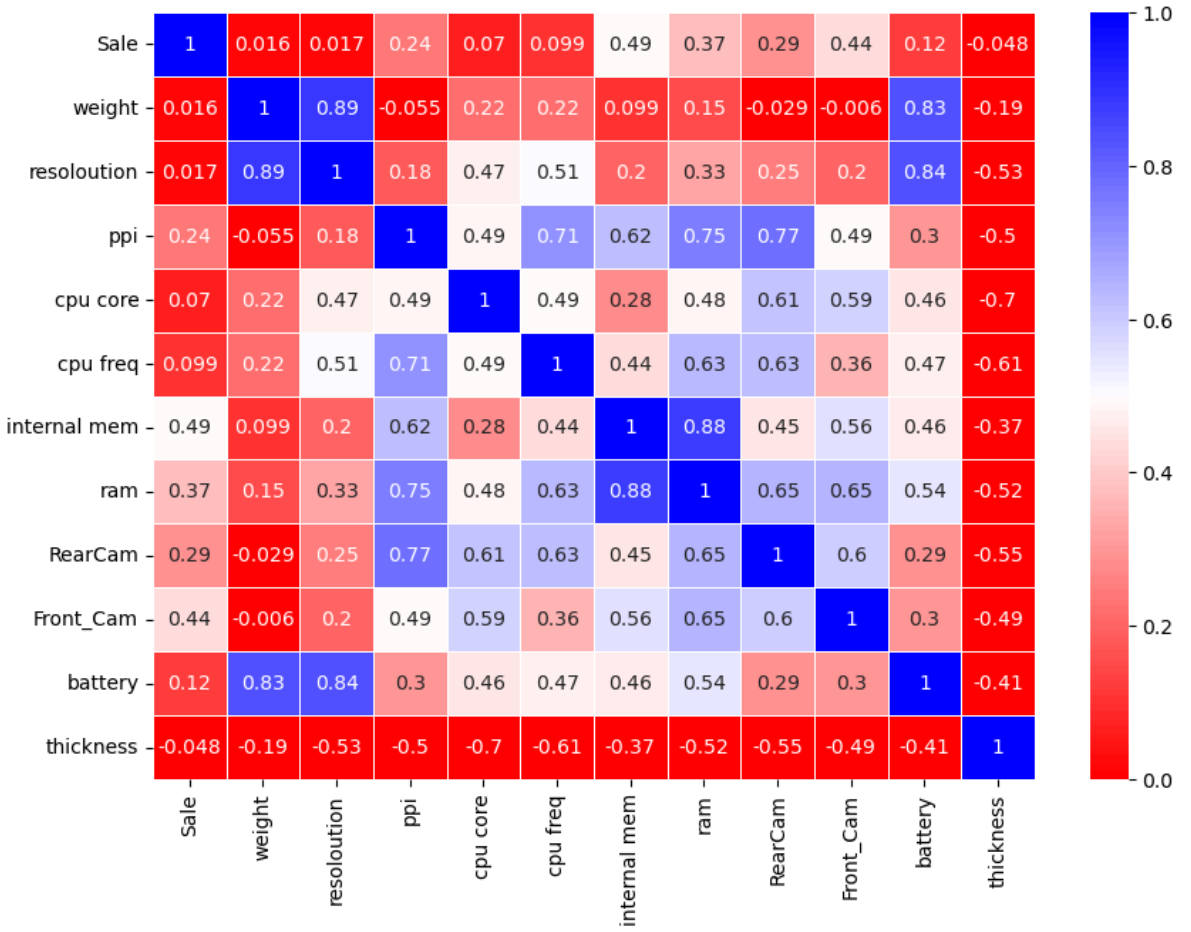
161 rows × 14 columns

In [24]:
```
X_cellphones = df_cellphones.iloc[:,2:]
y_cellphones = df_cellphones.iloc[:,1]
```

# Variables independientes e idénticamente distribuidas (i.i.d)

Las estimaciones de coeficientes para los mínimos cuadrados ordinarios se basan en la independencia de las características. Cuando las características están correlacionadas y las columnas de la matriz $X$ tienen una dependencia aproximadamente lineal, la matriz se vuelve casi singular y, como resultado, la estimación de mínimos cuadrados se vuelve muy sensible a los errores aleatorios en el objetivo observado, lo que produce una gran variación.

In [25]:
```
plt.figure(figsize=(10, 7))
sns.heatmap(X_cellphones.corr(), vmin=0, vmax=1, annot=True, cmap='bwr_r', linewidt
plt.show()
```




In [26]:
```
X_train_cellphones, X_test_cellphones, y_train_cellphones, y_test_cellphones = trai
```

In [27]:
```
regressor_cellphones = linear_model.LinearRegression()
regressor_cellphones.fit(X_train_cellphones, y_train_cellphones)
```

Out[27]:
```
▾ LinearRegression

LinearRegression()
```

In [28]:
```
print(f"MAE Linear Regression Cellphone Price Prediction: {mean_absolute_error(y_te
```

```
MAE Linear Regression Cellphone Price Prediction: 147.7
```

# Feature importance

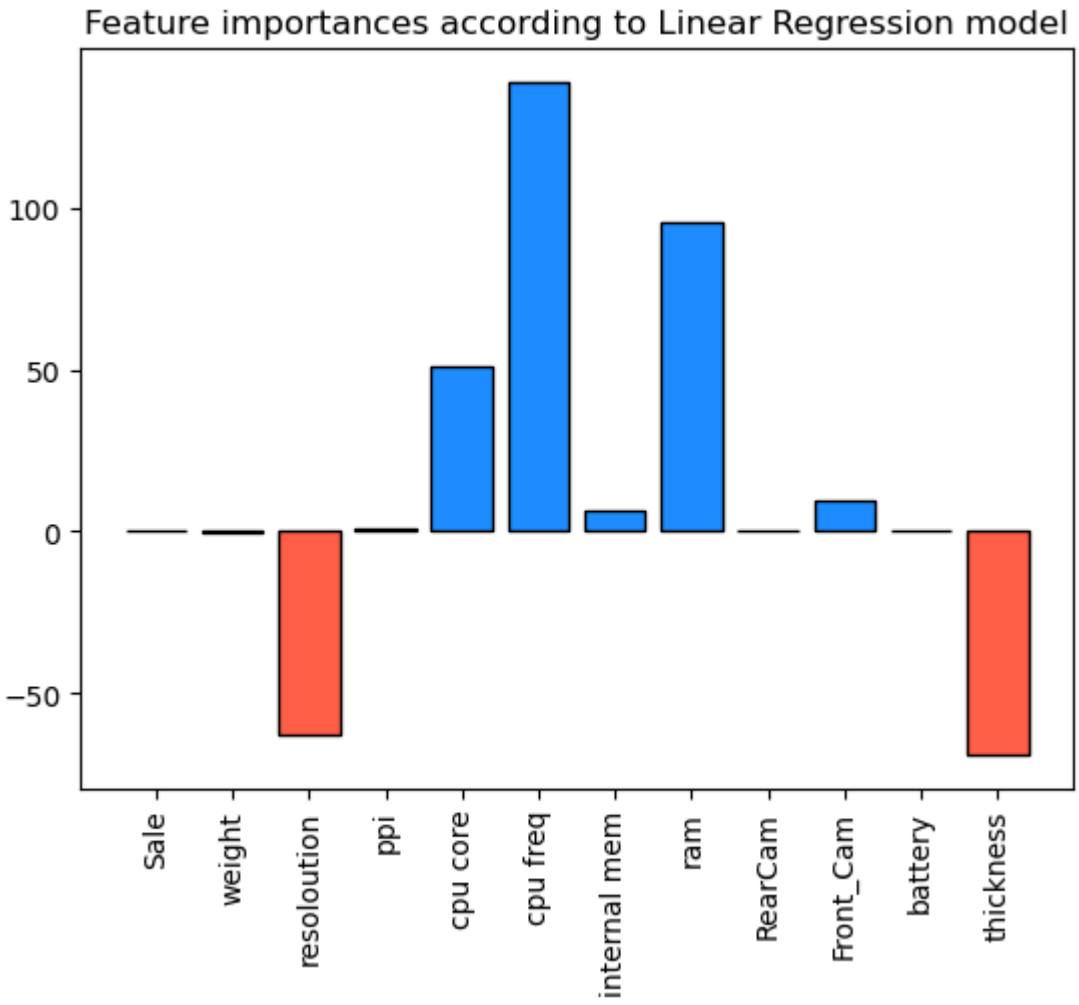
En los modelos lineales, el valor objetivo se modela como una combinación lineal de las características.

Los coeficientes representan la relación entre la característica dada $X_i$ y la variable objetivo $y$, asumiendo que todas las demás características permanecen constantes (dependencia condicional). Esto es diferente de graficar $X_i$ versus $y$, y ajustar una relación lineal: en ese caso, todos los valores posibles de las otras características se tienen en cuenta en la estimación (dependencia marginal).

In [29]:
```
print("Coeficientes: \n", regressor_cellphones.coef_)
```

```
Coeficientes:
 [-2.08100505e-02 -5.28968983e-01 -6.26377708e+01  1.12067692e+00
  5.11459740e+01  1.38789890e+02  6.17426037e+00  9.56996302e+01
  2.69640296e-01  9.25969214e+00  1.31424498e-01 -6.92438076e+01]
```

In [30]:
```
plt.bar(height=regressor_cellphones.coef_, x=np.array(X_cellphones.columns), edgecc
        color=['tomato' if coef < 0 else 'dodgerblue' for coef in regressor_cellphc
plt.title("Feature importances according to Linear Regression model")
plt.xticks(rotation = 90)
plt.show()
```




# Ridge regression

Ridge regression evita el sobreajuste de los modelos lineales al imponer una penalización en el tamaño de los coeficientes:

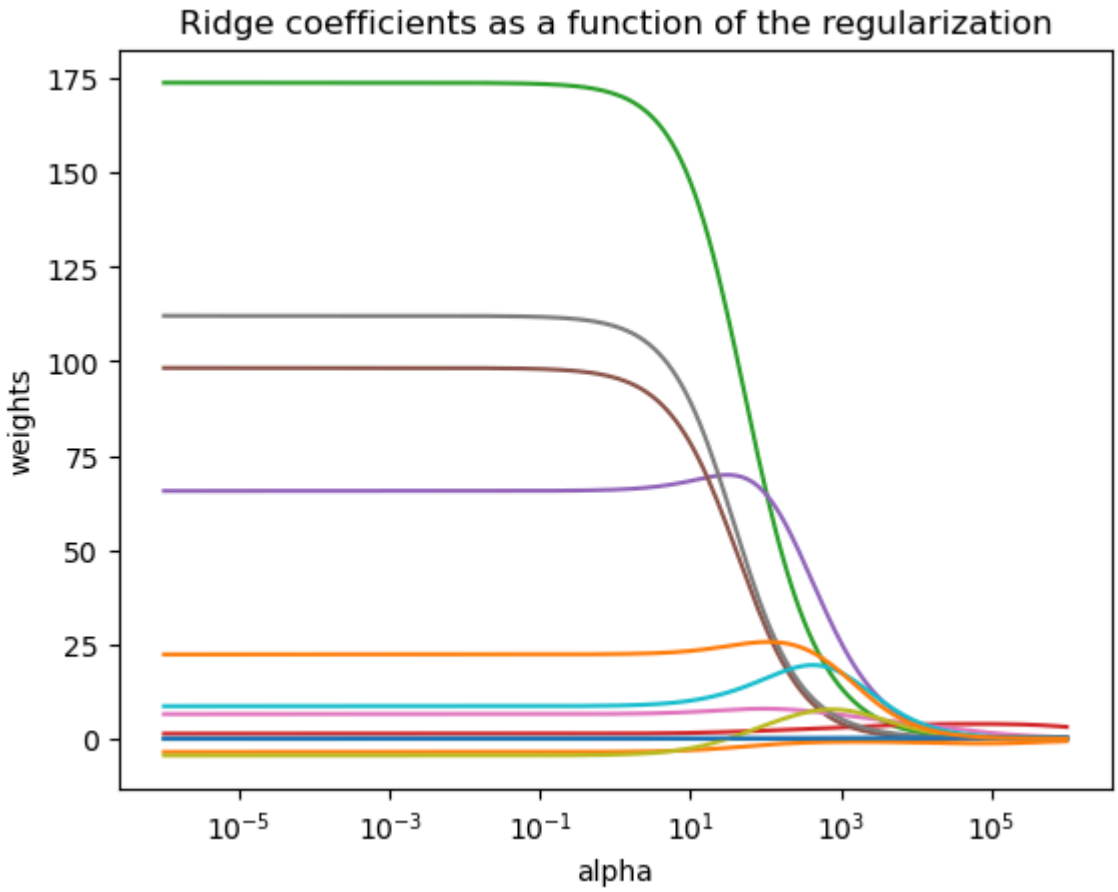
$$\min_w ||Xw - y||_2^2 + \alpha||w||_2^2$$

El parámetro de complejidad $\alpha \geq 0$ controla la cantidad de contracción: cuanto mayor sea el valor de $\alpha$, mayor será la cantidad de contracción y, por lo tanto, los coeficientes se vuelven más robustos a la colinealidad.

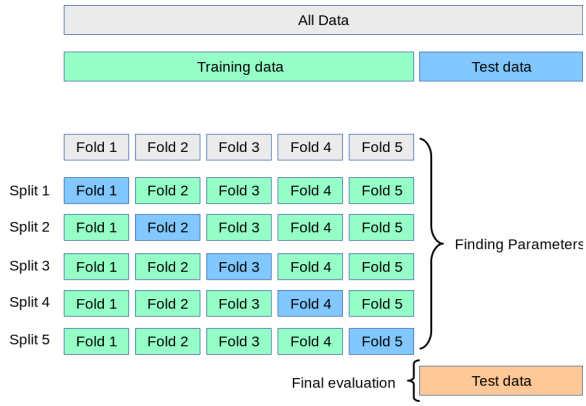
## Parámetro $\alpha$ en los coeficientes de Ridge

In [31]:
```
n_alphas = 200
alphas = np.logspace(-6, 6, n_alphas)
coefs = []
for a in alphas:
    r = linear_model.Ridge(alpha=a, fit_intercept=False)
    r.fit(X_train_cellphones, y_train_cellphones)
    coefs.append(r.coef_)
```

In [32]:
```
ax = plt.gca()
ax.plot(alphas, coefs)
ax.set_xscale("log")
ax.set_xlim(ax.get_xlim())
plt.xlabel("alpha")
plt.ylabel("weights")
plt.title("Ridge coefficients as a function of the regularization")
plt.axis("tight")
plt.show()
```




## Setting the regularization parameter: leave-one-out Cross-Validation




In [33]:
```
from sklearn.model_selection import cross_validate
from sklearn.model_selection import RepeatedKFold
```

In [34]:
```
ridge_cellphones = linear_model.RidgeCV(alphas=np.logspace(-6, 6, 13))
```

In [35]:
```
ridge_cellphones.fit(X_train_cellphones, y_train_cellphones)
```

Out[35]:

RidgeCV

```
RidgeCV(alphas=array([1.e-06, 1.e-05, 1.e-04, 1.e-03, 1.e-02, 1.e-01, 1.e
+00, 1.e+01,
       1.e+02, 1.e+03, 1.e+04, 1.e+05, 1.e+06]))
```

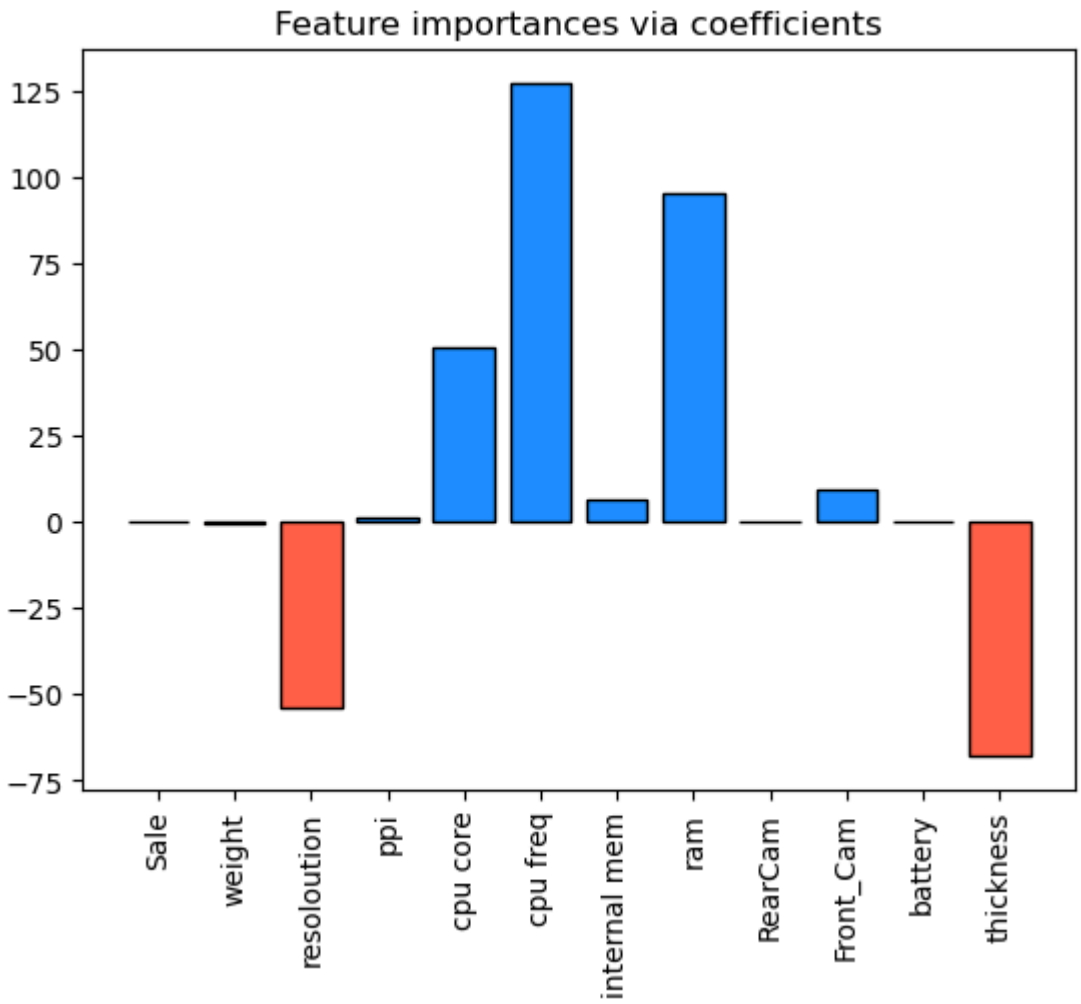
In [36]:
```
ridge_cellphones.alpha_
```

Out[36]:
```
1.0
```

In [37]:
```
print(f"MAE Ridge Cellphone Price Prediction: {mean_absolute_error(y_test_cellphone
```

```
MAE Ridge Cellphone Price Prediction: 147.9
```

In [38]:
```
plt.bar(height=ridge_cellphones.coef_, x=X_cellphones.columns, edgecolor='k',
        color=['tomato' if coef < 0 else 'dodgerblue' for coef in ridge_cellphones.
plt.title("Feature importances via coefficients")
plt.xticks(rotation = 90)
plt.show()
```




# Regresión polinomial

Es una extensión de los modelos lineales construyendo características polinómicas a partir de los coeficientes.

In [39]:
```
def true_fun(X):
    return np.cos(1.5 * np.pi * X)
```

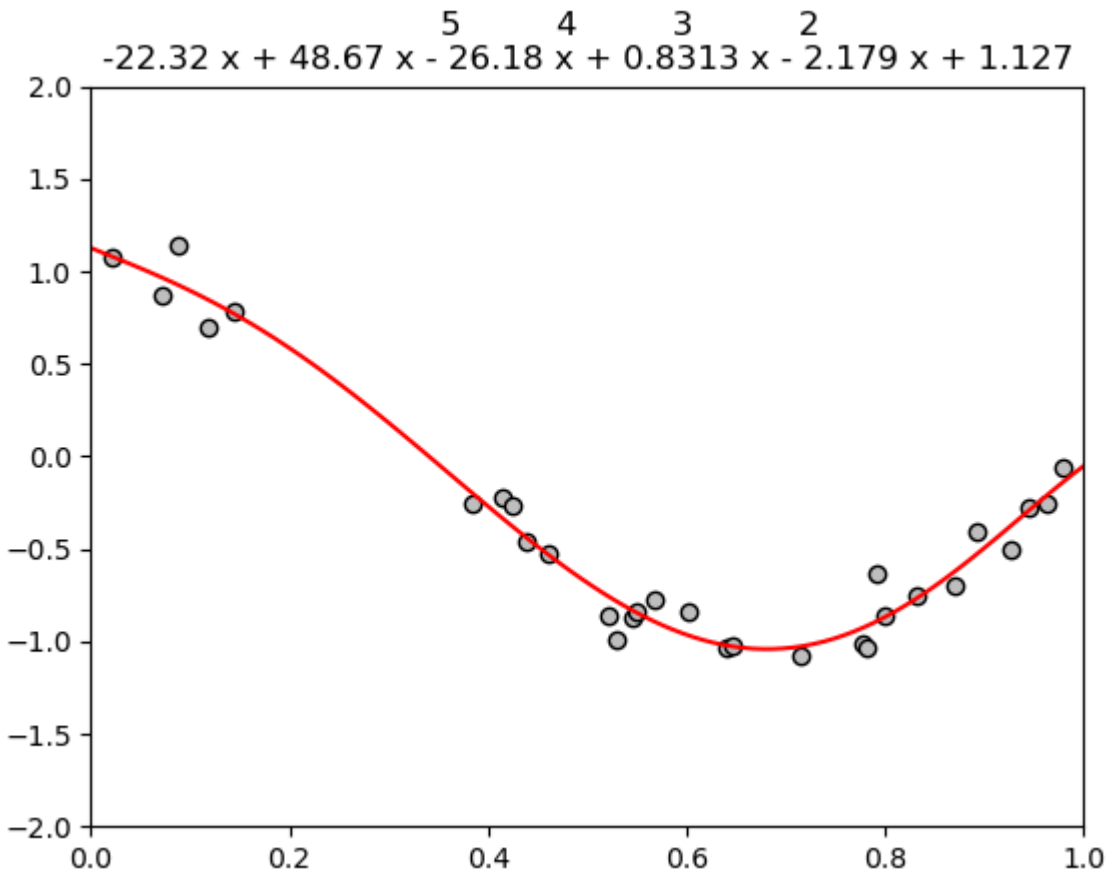
In [40]:
```
np.random.seed(0)

n_samples = 30
degrees = [1, 4, 15]

X = np.sort(np.random.rand(n_samples))
y = true_fun(X) + np.random.randn(n_samples) * 0.1
```

In [41]:
```
def polynomial_regression(degree):
    polyreg = np.poly1d(np.polyfit(X, y, degree))
    line = np.linspace(0, 1, 100)
    plt.scatter(X, y, edgecolor='k', color='silver')
    plt.plot(line, polyreg(line),c="r")
    plt.title(polyreg)
    plt.xlim((0, 1))
    plt.ylim((-2, 2))
    plt.show()
```

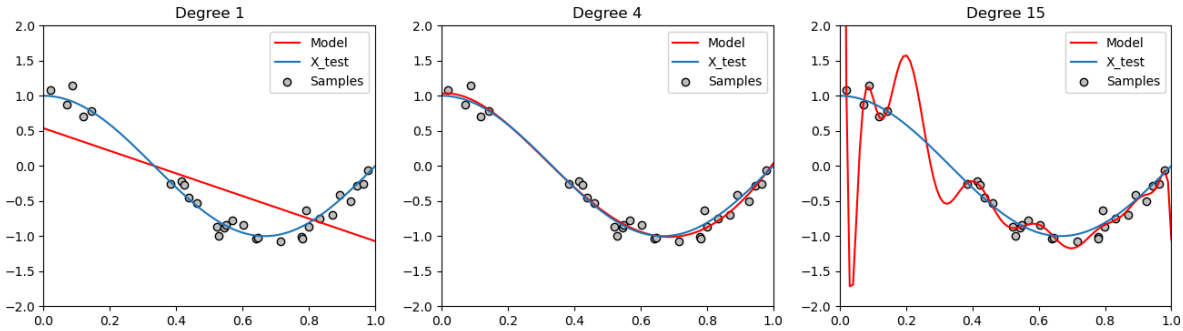
In [42]:
```
polynomial_regression(5)
```




# Underfitting vs. Overfitting

In [43]:
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
```

In [44]:
```
plt.figure(figsize=(16, 4))
for i in range(len(degrees)):
    ax = plt.subplot(1, len(degrees), i + 1)
    polynomial_features = PolynomialFeatures(degree=degrees[i], include_bias=False)
    linear_regression = linear_model.LinearRegression()
    pipeline = Pipeline([("polynomial_features", polynomial_features),("linear_regr
    pipeline.fit(X[:, np.newaxis], y)
    X_test = np.linspace(0, 1, 100)
    plt.plot(X_test, pipeline.predict(X_test[:, np.newaxis]),label="Model",c="r")
    plt.plot(X_test, true_fun(X_test), label="X_test")
    plt.scatter(X, y, edgecolor='k', color='silver', label="Samples")
    plt.xlim((0, 1))
    plt.ylim((-2, 2))
    plt.legend(loc="best")
    plt.title("Degree {}".format(degrees[i]))
plt.show()
```




In [45]:
```
# Definir una lista de grados a probar
degrees = [1, 2, 3, 4, 5]

# Almacenar los errores de validación para cada grado
validation_errors = []

for degree in degrees:
    poly = PolynomialFeatures(degree)
    X_train_poly = poly.fit_transform(X_train_cellphones)

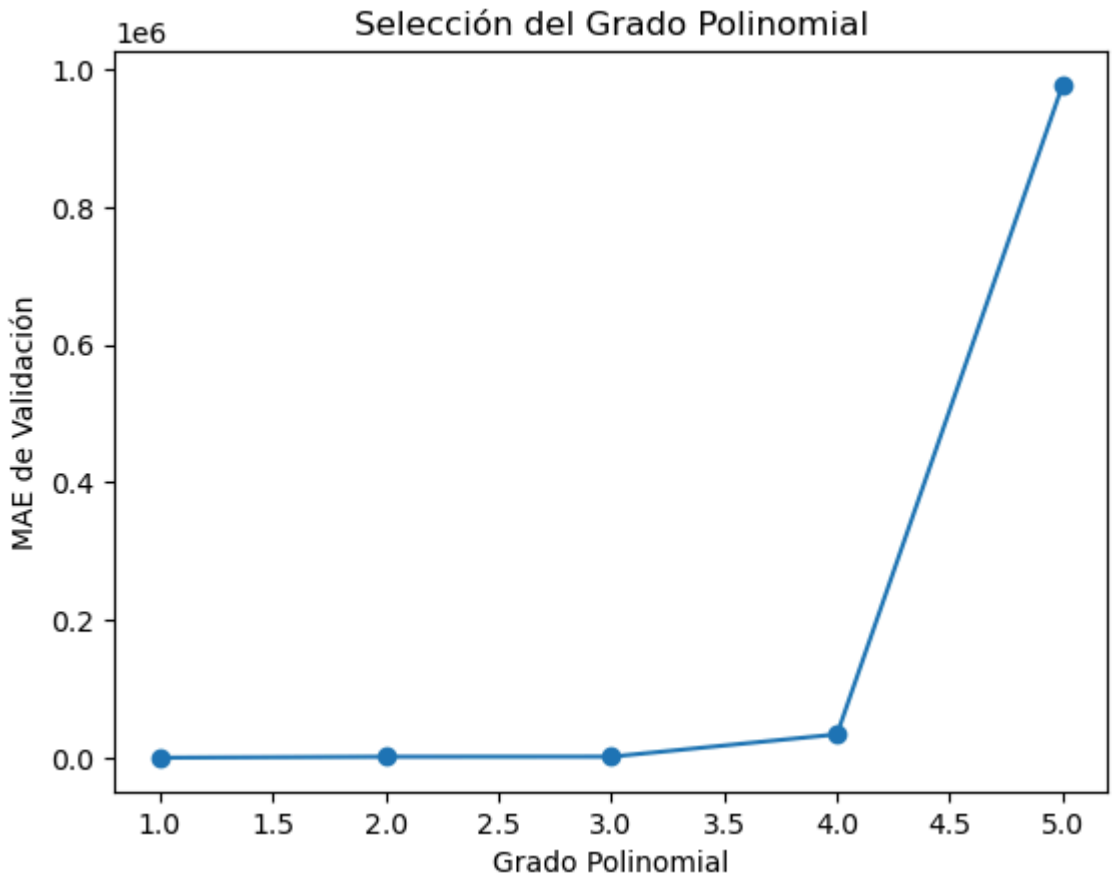
    # Crear un modelo de regresión lineal y realizar validación cruzada
    model = linear_model.LinearRegression()
    cv_scores = cross_val_score(model, X_train_poly, y_train_cellphones, cv=5, scor

    # Guardar el error promedio de la validación
    validation_errors.append(-np.mean(cv_scores))

# Determinar el grado con el menor error
best_degree = degrees[np.argmin(validation_errors)]
print(f"El grado óptimo basado en validación cruzada es: {best_degree}")
```

```
El grado óptimo basado en validación cruzada es: 1
```

In [46]:
```
plt.plot(degrees, validation_errors, marker='o')
plt.xlabel('Grado Polinomial')
plt.ylabel('MAE de Validación')
plt.title('Selección del Grado Polinomial')
plt.show()
```




In [47]:
```
# Transformación Polinomial (grado 1)
poly_features = PolynomialFeatures(degree=1)
X_train_poly = poly_features.fit_transform(X_train_cellphones)
X_test_poly = poly_features.transform(X_test_cellphones)

# Regresión Polinomial (ajuste usando la regresión lineal sobre características pol
regressor_poly = linear_model.LinearRegression()
regressor_poly.fit(X_train_poly, y_train_cellphones)
y_pred_poly = regressor_poly.predict(X_test_poly)
mae_poly = mean_absolute_error(y_test_cellphones, y_pred_poly)
print(f"MAE Polynomial Regression (degree 1) Cellphone Price Prediction: {mae_poly:
```

```
MAE Polynomial Regression (degree 1) Cellphone Price Prediction: 147.7
```

# RidgeClassifier

El regresor de Ridge tiene una variante clasificadora: RidgeClassifier. Este clasificador primero convierte los objetivos binarios en {-1, 1} y luego trata el problema como una tarea de regresión, optimizando el mismo objetivo que el anterior. La clase predicha corresponde al signo de la predicción del regresor. Para la clasificación multiclase, el problema se trata como una regresión de salida múltiple y la clase predicha corresponde a la salida con el valor más alto.

## Titanic dataset

El hundimiento del Titanic es uno de los naufragios más infames de la historia. El 15 de abril de 1912, durante su viaje inaugural, el RMS Titanic, ampliamente considerado como "insumergible", se hundió después de chocar con un iceberg. Desafortunadamente, no había

suficientes botes salvavidas para todos a bordo, lo que resultó en la muerte de 1502 de los 2224 pasajeros y tripulantes. Si bien hubo algún elemento de suerte involucrado en la supervivencia, parece que algunos grupos de personas tenían más probabilidades de sobrevivir que otros.

In [48]:
```
df_titanic = pd.read_csv('titanic.csv')
```

In [49]:
```
df_titanic
```

Out[49]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cal |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | N |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | N |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C1 |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | N |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **886** | 887 | 0 | 2 | Montvila, Rev. Juozas | male | 27.0 | 0 | 0 | 211536 | 13.0000 | N |
| **887** | 888 | 1 | 1 | Graham, Miss. Margaret Edith | female | 19.0 | 0 | 0 | 112053 | 30.0000 | E |
| **888** | 889 | 0 | 3 | Johnston, Miss. Catherine Helen "Carrie" | female | NaN | 1 | 2 | W./C. 6607 | 23.4500 | N |
| **889** | 890 | 1 | 1 | Behr, Mr. Karl Howell | male | 26.0 | 0 | 0 | 111369 | 30.0000 | C1 |
| **890** | 891 | 0 | 3 | Dooley, Mr. Patrick | male | 32.0 | 0 | 0 | 370376 | 7.7500 | N |

891 rows × 12 columns

In [50]:
```
# Survived: 0 = No, 1 = Si
# Pclass Ticket class: 1 = 1st, 2 = 2nd, 3 = 3rd
# SibSp: Número de hermanos / cónyuges a bordo del Titanic
# Parch: Nº de padres/hijos a bordo del Titanic
# Fare: Tarifa del pasajero
data_titanic = df_titanic.loc[:,('Survived','Pclass','Sex','Age','SibSp','Parch','F
```

In [51]:
```
data_titanic.dropna(inplace=True)
```

In [52]:
```
data_titanic
```

Out[52]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare |
|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | male | 22.0 | 1 | 0 | 7.2500 |
| **1** | 1 | 1 | female | 38.0 | 1 | 0 | 71.2833 |
| **2** | 1 | 3 | female | 26.0 | 0 | 0 | 7.9250 |
| **3** | 1 | 1 | female | 35.0 | 1 | 0 | 53.1000 |
| **4** | 0 | 3 | male | 35.0 | 0 | 0 | 8.0500 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **885** | 0 | 3 | female | 39.0 | 0 | 5 | 29.1250 |
| **886** | 0 | 2 | male | 27.0 | 0 | 0 | 13.0000 |
| **887** | 1 | 1 | female | 19.0 | 0 | 0 | 30.0000 |
| **889** | 1 | 1 | male | 26.0 | 0 | 0 | 30.0000 |
| **890** | 0 | 3 | male | 32.0 | 0 | 0 | 7.7500 |

714 rows × 7 columns

In [53]:
```
from sklearn.preprocessing import LabelEncoder
```

In [54]:
```
le = LabelEncoder()
data_titanic.Sex = le.fit_transform(data_titanic.Sex)
# male: 1 - female:0
```

In [55]:
```
data_titanic
```

Out[55]:

| | Survived | Pclass | Sex | Age | SibSp | Parch | Fare |
|---|---|---|---|---|---|---|---|
| **0** | 0 | 3 | 1 | 22.0 | 1 | 0 | 7.2500 |
| **1** | 1 | 1 | 0 | 38.0 | 1 | 0 | 71.2833 |
| **2** | 1 | 3 | 0 | 26.0 | 0 | 0 | 7.9250 |
| **3** | 1 | 1 | 0 | 35.0 | 1 | 0 | 53.1000 |
| **4** | 0 | 3 | 1 | 35.0 | 0 | 0 | 8.0500 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **885** | 0 | 3 | 0 | 39.0 | 0 | 5 | 29.1250 |
| **886** | 0 | 2 | 1 | 27.0 | 0 | 0 | 13.0000 |
| **887** | 1 | 1 | 0 | 19.0 | 0 | 0 | 30.0000 |
| **889** | 1 | 1 | 1 | 26.0 | 0 | 0 | 30.0000 |
| **890** | 0 | 3 | 1 | 32.0 | 0 | 0 | 7.7500 |

714 rows × 7 columns

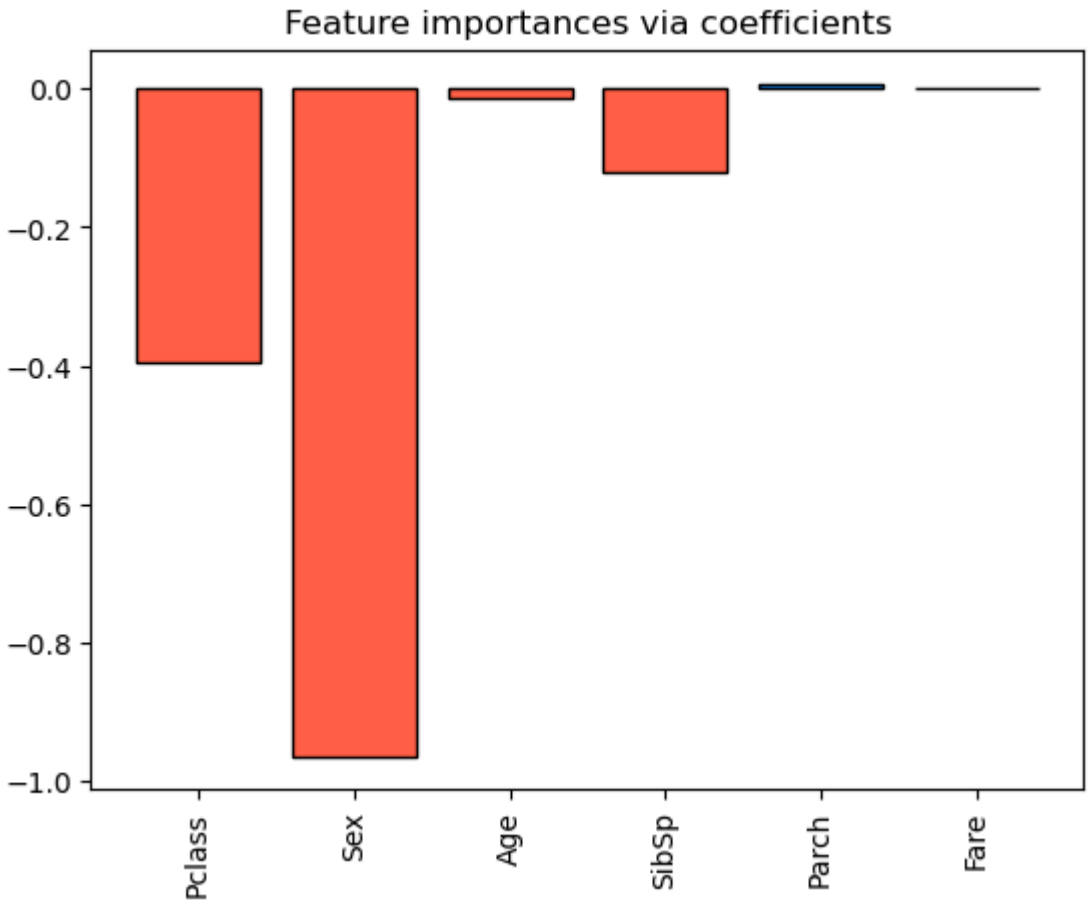
In [56]:
```
X_titanic = data_titanic.iloc[:,1:]
y_titanic = data_titanic.iloc[:,0]
```

In [57]:
```
X_train_titanic, X_test_titanic, y_train_titanic, y_test_titanic = train_test_split
```

In [58]:
```
from sklearn.linear_model import RidgeClassifier
```

In [59]:
```
ridge_clf_titanic = RidgeClassifier().fit(X_train_titanic, y_train_titanic)
```

In [60]:
```
plt.bar(height=(ridge_clf_titanic.coef_.tolist()[0]), x=X_titanic.columns, edgecolc
        color=['tomato' if coef < 0 else 'dodgerblue' for coef in ridge_clf_titanic
plt.title("Feature importances via coefficients")
plt.xticks(rotation = 90)
plt.show()
```




# Accuracy Score

Fracción de muestras clasificadas correctamente. El mejor rendimiento es 1.

In [61]:
```
print(f"Accuracy Training RidgeClassifier Titanic Dataset: {ridge_clf_titanic.score
print(f"Accuracy Testing RidgeClassifier Titanic Dataset: {ridge_clf_titanic.score(
```

```
Accuracy Training RidgeClassifier Titanic Dataset: 0.796
Accuracy Testing RidgeClassifier Titanic Dataset: 0.786
```

# Logistic regression

A pesar de su nombre, se implementa como un modelo lineal para clasificación en lugar de regresión. La regresión logística también se conoce en la literatura como regresión logit, clasificación de máxima entropía (MaxEnt) o clasificador log-lineal. En este modelo, las

probabilidades que describen los posibles resultados se modelan utilizando una función logística.

In [62]:
```
xmin, xmax = -5, 5
n_samples = 100
np.random.seed(0)
X = np.random.normal(size=n_samples)
y = (X > 0).astype(float)
X[X > 0] *= 4
X += 0.3 * np.random.normal(size=n_samples)
X = X[:, np.newaxis]
```

In [63]:
```
from scipy.special import expit
```

In [64]:
```
logreg = linear_model.LogisticRegression(C=1e5)
logreg.fit(X, y)
```

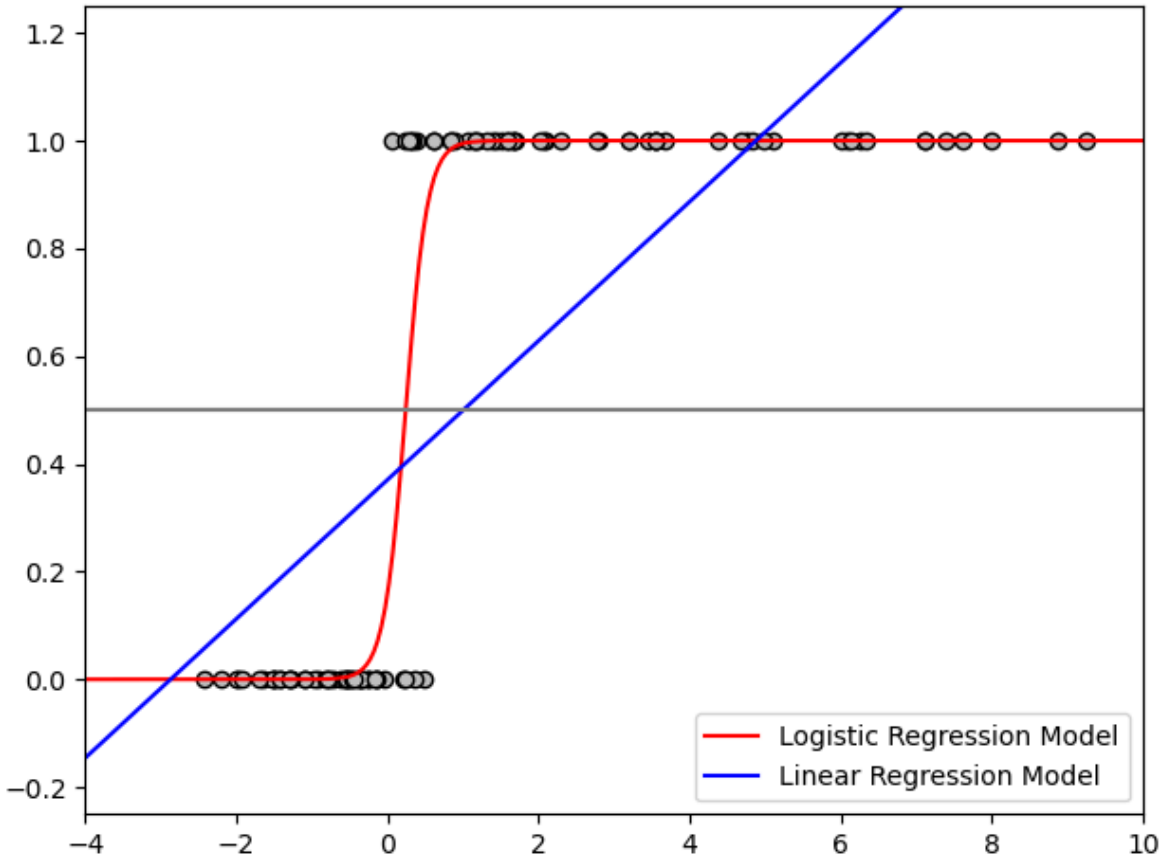
Out[64]:
```
LogisticRegression
LogisticRegression(C=100000.0)
```

In [65]:
```
X_test = np.linspace(-5, 10, 300)
loss = expit(X_test * logreg.coef_ + logreg.intercept_).ravel()
```

In [66]:
```
ols = linear_model.LinearRegression()
ols.fit(X, y)
```

Out[66]:
```
LinearRegression
LinearRegression()
```

In [67]:
```
plt.figure()
plt.scatter(X.ravel(), y,  edgecolor='k', color='silver')
plt.plot(X_test, loss, label="Logistic Regression Model", color="red")
plt.plot(X_test, ols.coef_ * X_test + ols.intercept_, label="Linear Regression Mode
plt.axhline(0.5, color=".5")
plt.ylim(-0.25, 1.25)
plt.xlim(-4, 10)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```




In [68]:
```
logreg_titanic = linear_model.LogisticRegression(max_iter=2000).fit(X_train_titanic
```

In [69]:
```
print(f"Accuracy RidgeClassifier Titanic Dataset: {ridge_clf_titanic.score(X_test_t
print(f"Accuracy LogisticRegression Titanic Dataset: {logreg_titanic.score(X_test_t
```

```
Accuracy RidgeClassifier Titanic Dataset: 0.786
Accuracy LogisticRegression Titanic Dataset: 0.795
```